

HBnB Technical Documentation

Introduction:

HBnB is an educational project for Holberton School.

The goal for this project is to build a simple website inspired by AirBnB using Python and the Flask framework. The codebase is only made with educational purposes in mind and it is not meant for a production environment. The focus is to teach fundamentals of software planning and architecture by recreating a real application from scratch including the backend and frontend.

The app is divided in three main layers

- Presentation Layer: This includes the services and API through which users interact with the system.
- Business Logic Layer: This contains the models and the core logic of the application.
- Persistence Layer: This is responsible for storing and retrieving data from the database.

The app should have the following functions:

- User Management: Users can register, update their profiles, and be identified as either regular users or administrators.
- Place Management: Users can list properties (places) they own, specifying details such as name, description, price, and location (latitude and longitude). Each place can also have a list of amenities.
- Review Management: Users can leave reviews for places they have visited, including a rating and a comment.
- Amenity Management: The application will manage amenities that can be associated with places.

Repository for this project: <https://github.com/titomundo/holbertonschool-hbnb>

High-Level Architecture:

The application is designed with a layer architecture in order to keep things simple. It is formed by three layers:

Presentation layer: The client side of the application, the purpose of this layer is to provide an interface for the client to interact with our app. It is composed of 3 parts:

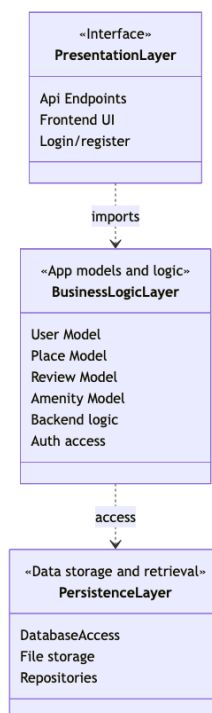
- API Endpoints
- Frontend UI
- Authentication.

Business Logic Layer: This is the backend of our application. It is in charge of the database models, authentication logic and token generation. It is composed of 6 parts:

- User Model
- Place Model
- Review Model
- Amenity Model
- Backend logic
- Auth access

Persistence Layer: This layer's goal is to create a database connection so we can have persistent data for our application. It is composed of 3 parts:

- Database Access
- File storage
- Repositories

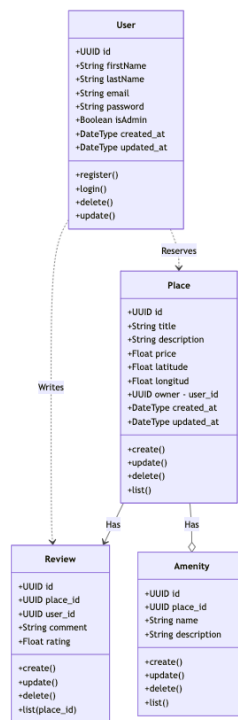


MermaidViewer.com

Business Logic Layer:

The project uses the following models to define the data used and the methods provided with each class. There are 4 classes:

- User: Use for authentication and storing user data such as name, email and ID.
- Place: Description of the place with location, prices, etc.
- Review: User-made reviews for a place, they are linked to the user who wrote the review.
- Amenity: Commodities that a place might have, a place can have many amenities.

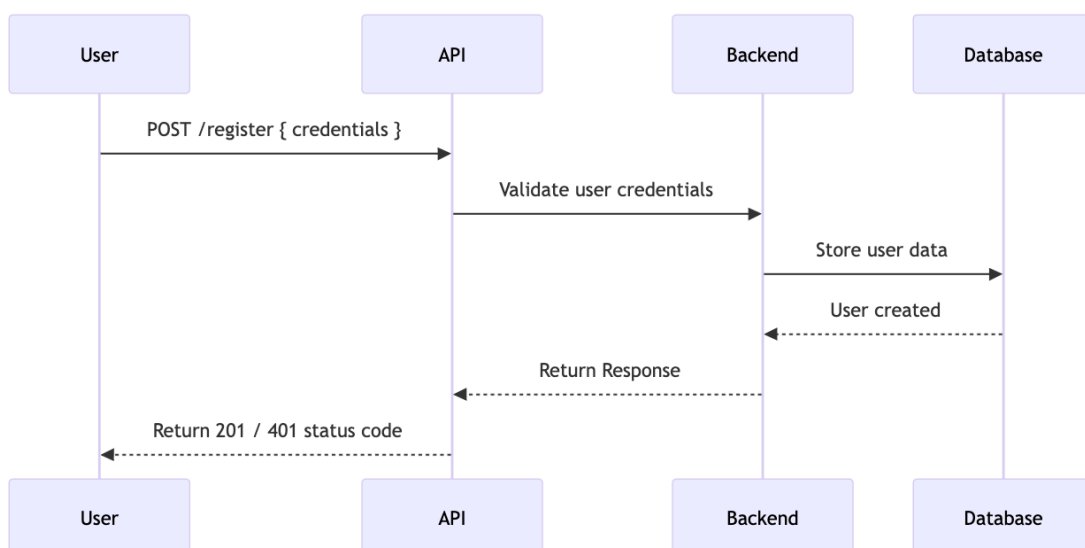


MermaidViewer.com

API Interaction Flow:

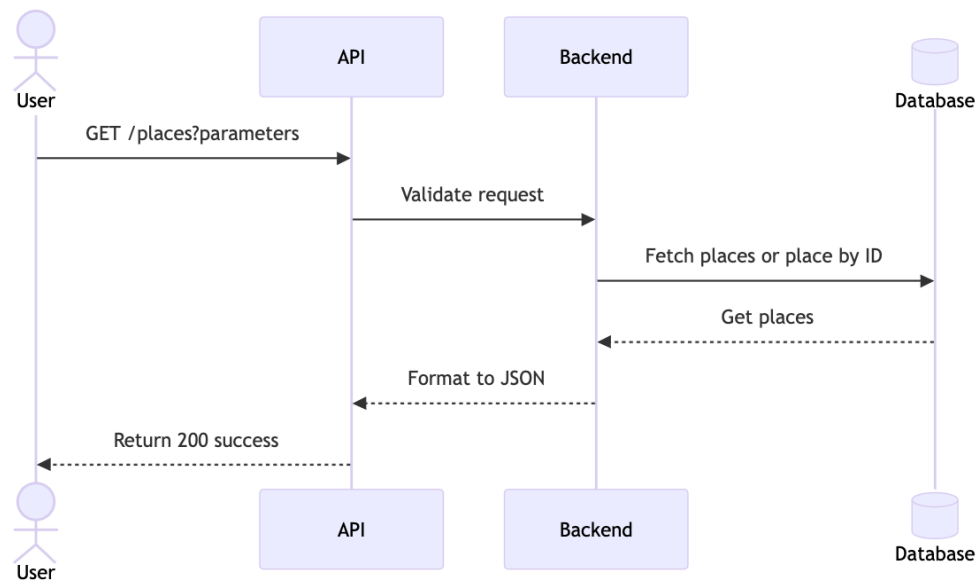
The API interaction for the user is divided into 4 main sequences: authentication, fetching places, creating places and creating reviews. These interactions are meant to be made through the frontend of the application.

A user registers in the app:



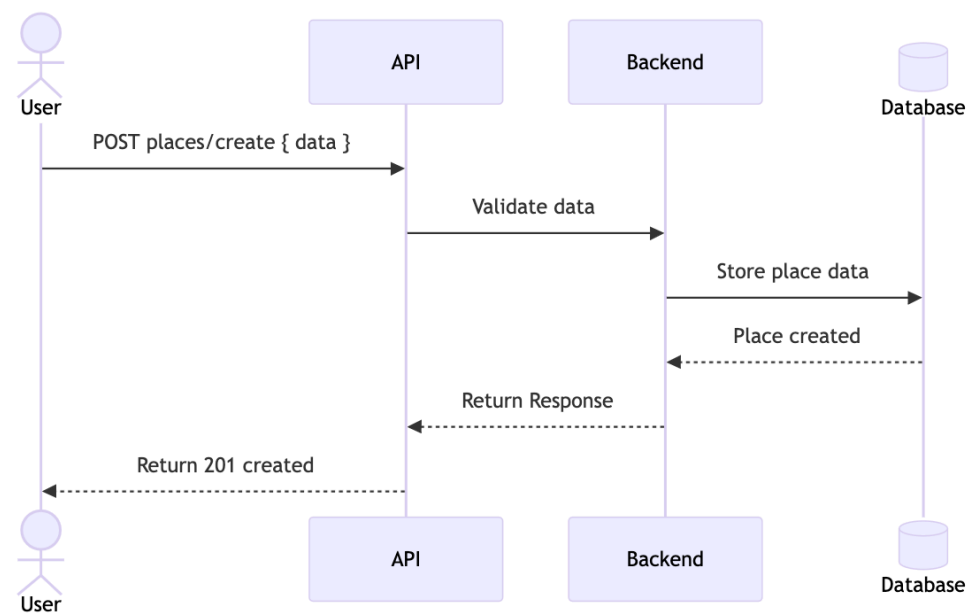
MermaidViewer.com

A user fetches places:



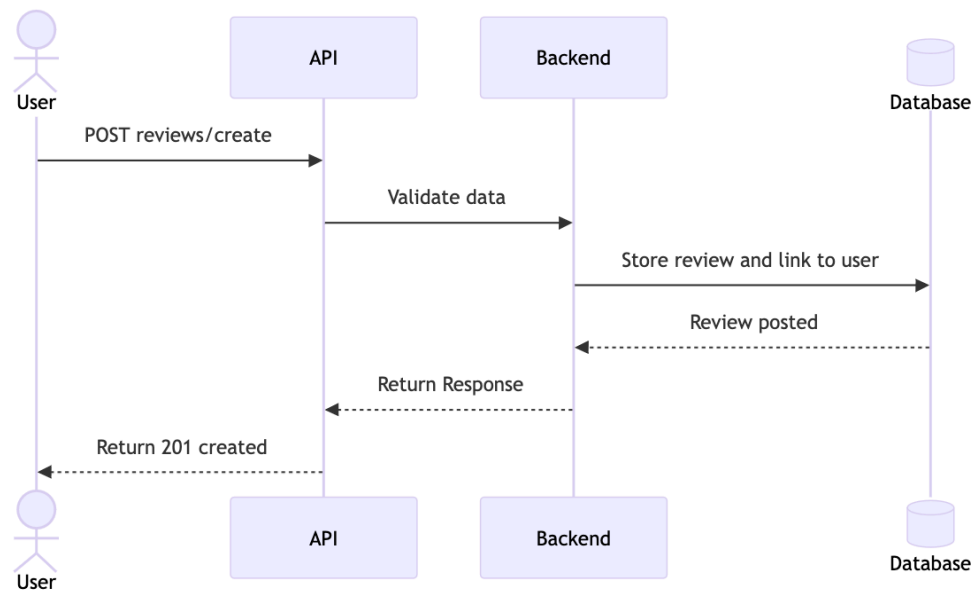
MermaidViewer.com

A user posts a place:



MermaidViewer.com

A user posts a review:



MermaidViewer.com